---

name: brief description: Produce a fresh living-brief snapshot for this repo via the brief-mcp server - carrying the previous document forward rewritten for the new situation, storing it in SQLite, rendering md+PDF, and updating the stable Git-Vista-current.pdf. Use after closing an issue, hitting a milestone or roadblock, before/after a workflow (pre-flights MUST be recorded here before launch), or whenever the user asks for a brief or the living doc.

The Living Brief

One brief per repo, served by brief-mcp (registered for both Claude accounts and codex). The owner reads exactly one path: `~/Documents/briefs/Git-Vista/Git-Vista-current.pdf` — the server overwrites it on every render of the latest snapshot.

Rules that are not optional

- **Fresh document each time**: read the prior snapshot, carry forward what still matters, REWRITE it for the new situation. Never an append log.
- **Real local timestamp** in the header AND the signature (`date --iso-8601=seconds`); the rendered filename carries it automatically.
- **Workflow pre-flights go IN the brief BEFORE launching** (owner's global rule): tier, agents, models, jobs, stopping condition, what it will NOT do. Keep them after the run, paired with the outcome.
- Standing sections ride forward: the command card, the honest milestone bars, the line counts, open threads.
- **NEVER hand-copy the milestone bars or the line counts. Regenerate both, every single brief, no exceptions:**

`bash milestone-bars --html # paste the block verbatim into the brief milestone-bars # the same numbers as a text table, for prose ./dev report # line counts by crate + repo health`

This is not a style preference. The bars were carried forward by hand across four briefs on 2026-08-05 and the owner asked, reasonably, "why is this milestone stuck at 61% for 12 hours?" They were in fact correct — but nobody could tell, because nothing had recomputed them, and a progress bar nobody recomputes is decoration. `milestone-bars` makes regenerating cheaper than copying, which is the only reason the habit will survive. It is one `gh` call and it does the one thing GitHub's own milestone UI cannot: split `COMPLETED` (shipped, counts) from `NOT_PLANNED` (cut, drawn outside the bar and out of the denominator), so removing scope never looks like finishing it. - **Say what moved since the last brief, with a from/to.** "61%" alone is unreadable; "48% → 61%, seven issues closed overnight, then flat ten hours" is the actual answer to the question the owner will ask. - Sign as your session account tag. Trigger field: freeform, honest.

Mechanics

1. `brief_current` (repo "Git-Vista") -> prior content + the id to pass as `parent_id`.

2. Compose the fresh full markdown.
3. `brief_generate` with content, parent_id, repo, trigger, author. A conflict reply means another agent published first: re-read, reconcile, retry with the new parent — never overwrite.
4. `brief_render` (repo "Git-Vista") -> timestamped md+pdf in history plus the `Git-Vista-current.pdf` slot update. Both the history file and the stable slot carry the repo in the FILENAME, not only the directory — a brief that leaves the folder still has to say which repo it is about.
5. Send the PDF to the owner when the update is one they asked about.